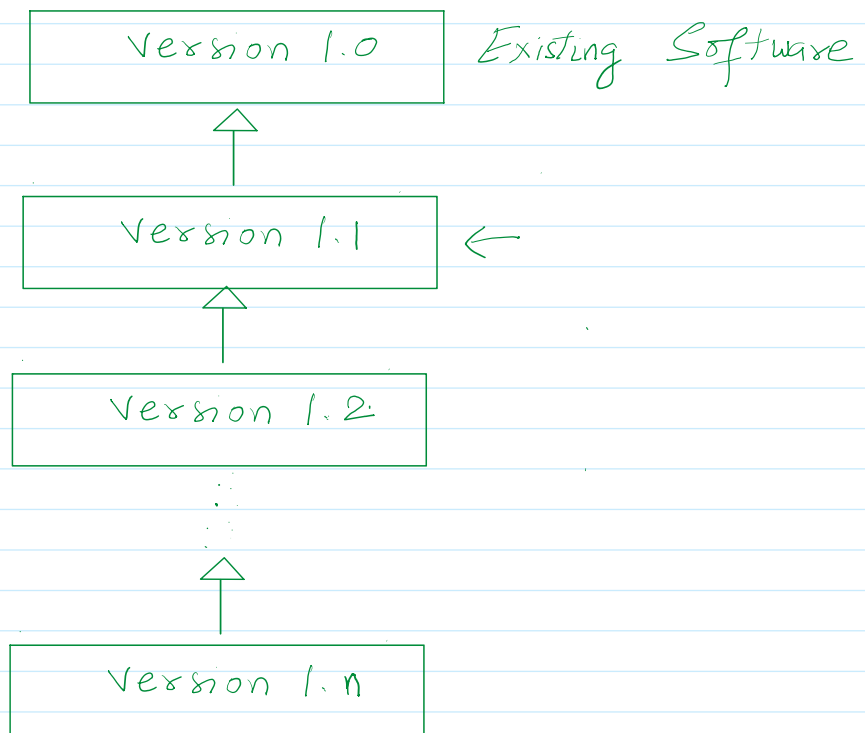


Python Inheritance

12 August 2026 06:43



Why inheritance is useful?

The software developer has to develop new classes, on the top of existing classes. The inheritance feature lets them to use the existing code. Therefore a new version of software can be written very easily.

Python Inheritance

Python inheritance allows one class to use the properties and methods of another class. Inheritance helps in reusing code and reducing repetition.

What is Inheritance in Python?

Inheritance is an important concept in **object-oriented programming** where one class (child class) inherits the properties and methods of another class (parent class)

Inheritance allows us to reuse the existing code instead of writing it again. The child class can use all features of the parent class and also add its own features or modify existing ones.

There are three main types of inheritance and Python:

1. Single Inheritance

- 2. Multiple Inheritance
- 3. Multilevel Inheritance

Base class / Parent class / Super class
↑
Derived class / child class / Sub-class

Implementing Inheritance

A child (derived) class Can inherit from parent (base) class by mentioning the parent class name inside parenthesis. This allows a child class to use the properties and methods of the parent class.

Syntax:

```
class DerivedClass(BaseClass):  
    #class body
```

The syntax of multiple inheritance:

```
class DerivedClass(Base1, Base2):  
    #class body
```

Example:

```
class Employee:  
  
    def display(self):  
        print("I am an employee")  
  
    def work(self):  
        print("Employee is working")
```

Now suppose we want to create:

- Manager
- Developer
- Tester

All of them are employees

```
class Employee:  
  
    def display(self):  
        print("I am an employee")  
  
    def work(self):  
        print("Employee is working")
```

```
class Manager:

    def display(self):
        print("I am a manager")

    def work(self):
        print("Manager is working")


class Developer:

    def display(self):
        print("I am a developer")

    def work(self):
        print("Developer is working")
```

This creates duplicate code.

Using Inheritance

```
class Employee:

    def display(self):
        print("I am an employee")

    def work(self):
        print("Employee is working")


class Manager(Employee):
    pass


class Developer(Employee):
    pass
```

Now Manager and Developer automatically receives the methods of Employee.

```
class Employee:

    def display(self):
        print("I am an employee")

    def work(self):
        print("Employee is working")


class Manager(Employee):
```

pass

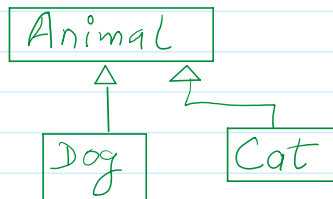
```
class Developer(Employee):  
    pass
```

#main code

```
anil = Manager()  
anil.display() } ← calling parent class method.  
anil.work()  
Trisha = Developer()  
Trisha.display()  
Trisha.work()
```

Output:

```
I am an employee  
Employee is working  
I am an employee  
Employee is working
```



```
class Animal:
```

```
    def eat(self):  
        print("Animal is eating")
```

```
class Dog(Animal):  
    def bark(self):  
        print("Dog is barking")
```

```
class Cat(Animal):  
    def meao(self):  
        print("Cat is saying meao...meao")
```

```
happy = Dog()  
happy.eat()  
happy.bark()  
poppins = Cat()  
poppins.eat()  
poppins.meao()
```

Output:

Animal is eating
Dog is barking
Animal is eating
Cat is saying meao...meao

The child class can access:

1. Its own methods
2. Inherited method

```
class Person:
    def set_person(self, name, age):
        self.name = name
        self.age = age

    def display_person(self):
        print(f"Name: {self.name}")
        print(f"Age: {self.age}")

class Student(Person):
    def set_student(self, roll_no, course):
        self.roll_no = roll_no
        self.course = course

    def display_student(self):
        print(f"Roll No. : {self.roll_no}")
        print(f"Course: {self.course}")

# main code
s = Student()
s.set_person('Vaanya', 12)
s.set_student(101, 'Python')
s.display_person()
s.display_student()
```

Output:

Name: Vaanya
Age: 12
Roll No. : 101
Course: Python

Inheritance with `__init__()`

Constructors are particularly important when using inheritance:
Consider:

```
class Person:
```

A constructor is a special method that is automatically
 # called when an object of a class is created.
 # It is mainly used to initialize the attributes of the object.
 # The constructor is defined using the `__init__()` method. It takes
 # `self` as a first parameter and can expect additional arguments
 # to set values.

```
def __init__(self, name, age):
    self.name = name
    self.age = age
```

```
class Student(Person):
```

```
def __init__(self, name, age, roll_no, course):
    # super() is used to access members of the parent class.
    super().__init__(name, age)
    # self.name = name
    # self.age = age
    self.roll_no = roll_no
    self.course = course
```

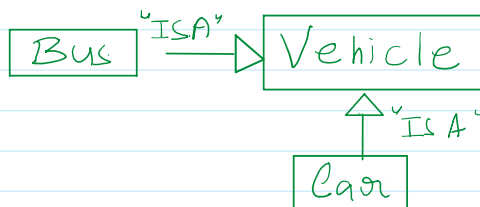
calls

First parent class object is created.

After that child class object is created.

```
s = Student("Rahul", 21, 101, "C++")
print(s.name) # parent class instance variable
print(s.course) # child class instance variable
```

Output:
 Rahul
 C++



Method Inheritance

A child class automatically inherits methods from the parent.

```
class Vehicle:
    def start(self):
        print("vehicle started")

    def stop(self):
        print("vehicle stopped")

class Car(Vehicle):
    def drive(self):
        print("Car is driving")
```

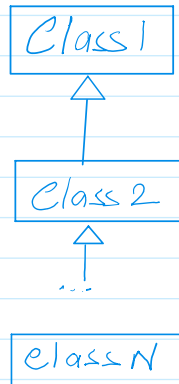
```
# main code
toyotacar = Car()
toyotacar.start()
toyotacar.drive()
toyotacar.stop()
```

Output:

vehicle started
Car is driving
vehicle stopped

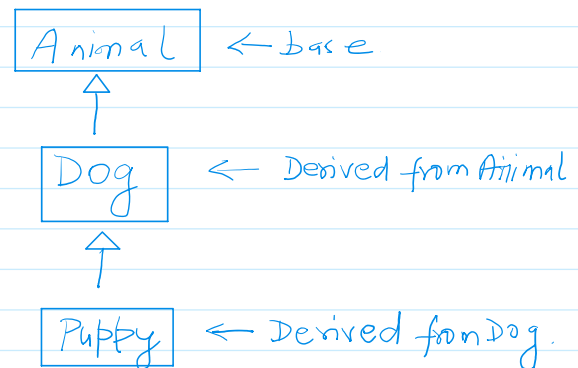
Python Multi-level Inheritance

Multi - level inheritance occurs when a class inherits from another class. This creates a chain of inheritance. Python allows multiple levels of inheritance without any fixed limit.



Syntax:

```
class Class1:  
    #base class  
  
class Class2(Class1):  
    # derived from class1  
  
class Class3(Class2):  
    # derived from Class2
```



Code:

```
class Animal:  
    def eat(self):  
        print("Animal is eating")  
  
class Dog(Animal):  
    def bark(self):  
        print("Dog is barking")  
  
class Cat(Animal):  
    def meao(self):  
        print("Cat is saying meao...meao")  
  
class Puppy(Dog):
```

```

def cryForMilk(self):
    print("Puppy crying for milk...")

happy = Dog()
happy.eat()
happy.bark()
poppins = Cat()
poppins.eat()
poppins.meao()
troy = Puppy()
troy.cryForMilk()
troy.bark()
troy.eat()

```

Output:

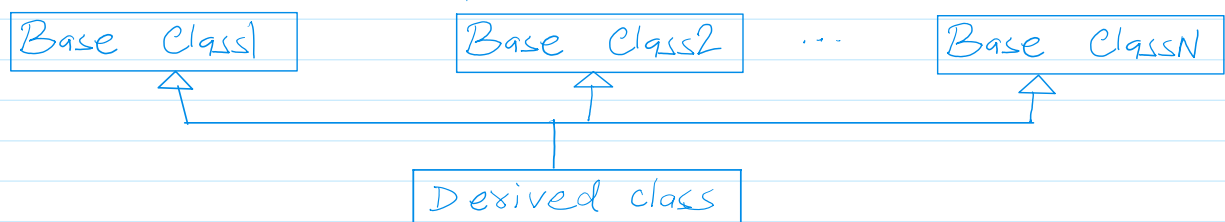
```

Animal is eating
Dog is barking
Animal is eating
Cat is saying meao...meao
Puppy crying for milk...
Dog is barking
Animal is eating

```

Python Multiple Inheritance

Multiple inheritance allows a class to inherit from more than one parent class. This means a child class can use the properties and methods of multiple base classes.



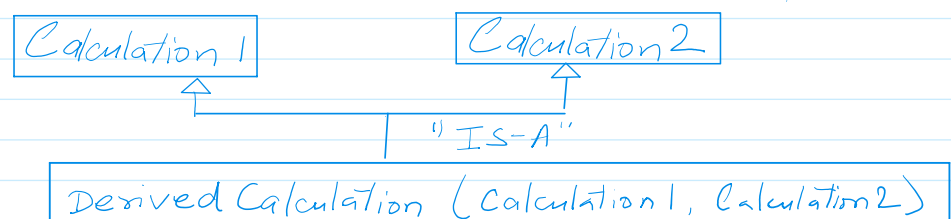
Syntax:

```

class DerivedClass(Base1, Base2,...):
    # class body

```

For example:



Derived Calculation (Calculation1, Calculation2)

```

class Calculation1:
    def sum(self, a, b):
        return a + b

class Calculation2:
    def multiplication(self, a, b):
        return a * b

class DerivedCalculation(Calculation1, Calculation2):
    def divide(self, a, b):
        return a / b

d = DerivedCalculation()
print(d.sum(8, 10))
print(d.multiplication(8, 10))
print(d.divide(8, 10))

```

Output:

```

18
80
0.8

```

Checking class Relationship

In Python we can check whether one class is derived from another using **issubclass()** function. It returns true if the first class is a subclass of the 2nd class.

Code:

```

class Calculation1:
    def sum(self, a, b):
        return a + b

class Calculation2:
    def multiplication(self, a, b):
        return a * b

class DerivedCalculation(Calculation1, Calculation2):
    def divide(self, a, b):
        return a / b

d = DerivedCalculation()
print(d.sum(8, 10))
print(d.multiplication(8, 10))
print(d.divide(8, 10))

```

```
print(issubclass(DerivedCalculation, Calculation2))#true
print(issubclass(Calculation1, Calculation2)) #false
```

Output:

```
18
80
0.8
True
False
```

Checking Object and Class Relationship We can check whether an object belongs to a particular class using the `isinstance()` function. It returns *True* if the object is an instance of the given class, otherwise it returns *False*.

```
class Calculation1:
    def sum(self, a, b):
        return a + b

class Calculation2:
    def multiplication(self, a, b):
        return a * b

class DerivedCalculation(Calculation1, Calculation2):
    def divide(self, a, b):
        return a / b

d = DerivedCalculation()
print(d.sum(8, 10))
print(d.multiplication(8, 10))
print(d.divide(8, 10))
print(issubclass(DerivedCalculation, Calculation2))#true
print(issubclass(Calculation1, Calculation2)) #false
print(isinstance(d, DerivedCalculation)) #True
print(isinstance(d, Calculation2)) #True
print(isinstance(d, Calculation1)) #True
```

Output:

```
18
80
0.8
True
False
True
True
True
```

Redefining the `__init__()` Method

We can redefine the `__init__()` method in a child class. This allows the child class to have its own way of initializing object attributes. However, the parent class constructor is not called automatically, so we need to use `super()` if we want to include it.

Syntax:

```
class ChildClass(ParentClass):  
    def __init__(self, parameters):  
        super().__init__(parameters)  
        # additional code
```

Code:

```
class Person:  
  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age  
  
    def display_person(self):  
        print(f"Name: {self.name}")  
        print(f"Age: {self.age}")  
  
class Student(Person):  
    def __init__(self, roll_no, course, name, age):  
        super().__init__(name, age)  
        #additional code  
        self.roll_no = roll_no  
        self.course = course  
  
    def display_student(self):  
        print(f"Roll No. : {self.roll_no}")  
        print(f"Course: {self.course}")  
  
# main code  
s = Student(101, 'Python', 'Vaanya', 12)  
s.display_person()  
s.display_student()
```

Output:

```
Name: Vaanya  
Age: 12  
Roll No. : 101  
Course: Python
```

Abstraction in Python

Abstraction in Python is used to hide complex details and show only the important features of a program.

What is Abstraction in Python?

Abstraction is a concept in object oriented programming where we hide the internal implementation and show only the required functionality to the

user.

In Python, abstraction is achieved using abstract classes and abstract methods with the help of the **ABC** module.

Real Life example: In a payment system, different payment methods are available such as credit card UPI and net banking. All method follow a common process, but their internal working is hidden:

- **Common data:** amount, user details, transaction ID
- **Common actions:** pay(), validate(), generateReceipts()

Each method works differently-Credit Card uses card details, UPI used UPI ID, and Net Banking uses bank login credentials.

The system only knows that payment can be made and verified. How each payment method processes the transaction is hidden from the user.

Abstract Class

An abstract base class is a class that contains one or more abstract methods and objects cannot be created directly. These methods do not have any implementation and act as a blueprint for child classes.

To create an abstract base class, we import **ABC** and **abstractmethod** from the **ABC** module. The class must inherit from ABC and abstract methods are defined using the **@abstractmethod** decorator.

Code:

```
from abc import ABC, abstractmethod
#abstract base class
class Vehicle(ABC): ← abstract base class
    @abstractmethod
    def start(self):
        pass

    @abstractmethod
    def stop(self):
        pass

#child class
class Car(Vehicle):
    # Abstract methods are always implemented inside a child class.
    # This is called method overriding.
    def start(self):
        print("Car is starting with a key ignition.")

    def stop(self):
        print("Car is stopping using the brake")
```

```
my_car = Car()
my_car.start()
my_car.stop()
```

} → We cannot create an object of vehicle,
but we can create an object of car.
↑

```
my_car.start()
my_car.stop()
```

→ we cannot create an object of vehicle,
but we can create an object of car.

Output:

Car is starting with a key ignition.
Car is stopping using the brake

Rest of the code is written above:

```
my_vehicle = Vehicle()
my_vehicle.start()
my_vehicle.stop()
```

Output:

```
line 27, in <module>
    my_vehicle = Vehicle()
TypeError: Can't instantiate abstract class Vehicle without an
implementation for abstract methods 'start', 'stop'
```

Homework:

Inheritance in Python, covering single, multiple, multilevel, hierarchical, and hybrid inheritance, along with method overriding and super().

1. University Management System (Single Inheritance)

Design a base class called Person with attributes such as name, age, and gender, along with a method display_details(). Create a derived class Student that inherits from Person and adds specific attributes like student_id, course, and gpa.

Requirements:

- Override display_details() in the Student class using super() to display both general personal information and academic records.
- Add a method calculate_grade() in Student that returns letter grades based on gpa.
- Instantiate two Student objects and demonstrate all inherited and unique methods.

2. E-Commerce Product Catalog (Hierarchical Inheritance)

Create a base class Product with common attributes: product_id, name, price, and stock_quantity. Derive two specialized classes from Product:

- i. Electronics: Adds brand, warranty_period_months, and power_consumption.
- ii. Clothing: Adds size, material, and color.

Requirements:

- Implement a apply_discount(percentage) method in the base class.
- Override display_info() in both derived classes to include product-specific details.
- Include stock validation logic: write a method purchase(quantity) in Product that reduces stock and prints an invoice, raising a warning if

stock is insufficient.

3. Vehicle Fleet Tracking (Multilevel Inheritance)

Build a 3-tier hierarchy representing transport vehicles:

1. **Base Class:** Vehicle – Attributes: make, model, year, base_rental_rate.
2. **Intermediate Class:** Car (inherits from Vehicle) – Attributes: num_doors, fuel_type.
3. **Derived Class:** ElectricCar (inherits from Car) – Attributes: battery_capacity_kwh, range_km.

Requirements:

- Override a method calculate_rental_cost(days) at each tier:
 - Vehicle calculates flat daily rate.
 - Car adds a fixed fee for maintenance.
 - ElectricCar applies a 10% eco-friendly discount on the total rate.
- Use super() chaining across all constructors (__init__).

4. Smart Home System (Multiple Inheritance & Diamond Problem)

Demonstrate multiple inheritance using smart devices:

- Class Camera: Attributes (resolution, fps), method record_video().
- Class Speaker: Attributes (volume_level, sound_quality), method play_audio().
- Class SmartAssistantDevice (inherits from **both** Camera and Speaker): Attributes (device_id, assistant_name).

Requirements:

- Initialize attributes using super() properly across the inheritance graph.
- Print the Method Resolution Order (MRO) using SmartAssistantDevice. __mro__ or .mro() to explain how Python resolves conflicts.
- Implement a trigger_alert(message) method in SmartAssistantDevice that utilizes functionality from both parent classes.

5. Bank Account Types & Interest System (Method Overriding)

Create a base class BankAccount containing account_number, account_holder, and balance.

Requirements:

- Add methods deposit(amount) and withdraw(amount).
- Create a child class SavingsAccount that adds an interest_rate attribute and a method add_interest().
- Create another child class CheckingAccount that adds an overdraft_limit attribute and overrides withdraw() to allow withdrawals past zero up to the overdraft limit.
- Create a script that manages a list of mixed BankAccount objects and performs monthly processing (adding interest or applying overdraft fees dynamically).

6. Corporate Payroll System (Polymorphism & Inheritance)

Design a base class Employee with attributes employee_id, name, and base_salary, along with an abstract/placeholder method

`calculate_monthly_pay()`.

Requirements:

- Derive three types of employees:
 1. `FullTimeEmployee`: Gets `base_salary` plus fixed performance bonuses.
 2. `PartTimeEmployee`: Works on `hourly_rate` and `hours_worked`.
 3. `CommissionEmployee`: Gets base salary plus a commission percentage on `sales_volume`.
- Write a function `generate_payroll(employee_list)` that iterates through a list of diverse employee instances and displays their formatted pay stubs using polymorphic method calls.

7. RPG Game Character System (Hybrid Inheritance)

Design a character system for a game using a combination of multilevel and multiple inheritance:

1. Base class `Character`: `name`, `health`, `mana`.
2. Derived classes `Warrior` (adds `shield_power`, `melee_attack()`) and `Mage` (adds `spell_power`, `cast_spell()`).
3. Class `Paladin` (inherits from **both** `Warrior` and `Mage`): Combines physical and magical abilities.

Requirements:

- Resolve initialization using `super()` or explicit parent calling to avoid duplicate `Character` initialization.
- Implement a `battle_stance()` method in `Paladin` that temporarily boosts both `shield_power` and `spell_power`.
- Create sample combat rounds showing dynamic health reduction and mana usage.

8. Library Resource Management (Abstract Base Classes & Inheritance)

Use the `abc` module to define an abstract base class `LibraryItem`:

- Abstract methods: `check_out()`, `return_item()`, and `calculate_late_fee(days_overdue)`.
- Common properties: `item_id`, `title`, `is_checked_out`.

Requirements:

- Implement two concrete subclasses: `Book` and `DVD`.
 - `Book` late fee: \$0.50 per day.
 - `DVD` late fee: \$2.00 per day.
- Enforce abstraction rules: verify that instantiating `LibraryItem` directly raises an error, and ensure all subclasses implement all abstract methods properly.

9. Hospital Patient & Medical Record System (Encapsulation + Inheritance)

Develop a medical management program balancing encapsulation and inheritance:

- Base class `Patient`: Private attributes `__patient_id`, `__medical_history` (list of strings), and public/protected attributes `name` and `age`.
- Provide getter and setter methods to securely modify `__medical_history`.
- Subclass `InPatient`: Adds `room_number`, `admission_date`, and `daily_room_charge`.
- Subclass `OutPatient`: Adds `appointment_date` and `consultation_fee`.

Requirements:

- Implement `generate_bill(days)` in `InPatient` and `generate_bill()` in `OutPatient`.
- Ensure child classes cannot access private members of `Patient` directly, but instead use protected/public accessors.

10. Automated Notification Engine (Composition vs. Inheritance)

Design a system comparing inheritance with composition for sending multi-channel notifications:

- Base class `Notification`: `sender_id`, `timestamp`. Method `send(recipient, message)`.
- Subclasses: `EmailNotification` (adds `subject`, `email_server`) and `SMSNotification` (adds `phone_number`, `carrier_gateway`).

Requirements:

- Override `send()` in both subclasses to print realistic log outputs for emails and text messages.
- Create a higher-level class `NotificationManager` that uses **composition** (holds a list of different `Notification` instances) to dispatch bulk alerts across multiple channels simultaneously.